

# MASPOE's

# MVP Poll application

*3th BSc Business Engineering*  
*Database systems & Data Structures*

# OVERVIEW

01

---

About Maspoe

02

---

Why this project?

03

---

Bussines Idea

04

---

User journey

05

---

Core features

06

---

Demonstration



# Our Team



Judith  
Ceuppens  
General  
Manager



Louis  
Van Linden  
Backend  
Engineer



Arjen  
Sap  
Database  
Architect



Korneel  
Pennewaert  
Frontend  
Developer



Kasper  
De Vos  
Database  
Architect

# ABOUT MASPOE

- Belgian event-organizing organization
- Focused on sports, culture and entertainment
- Organises lots of events/festivals
- Biggest event: Vicaris Village



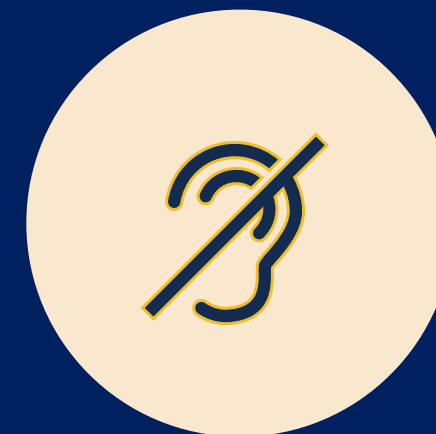
# Why this project?



Festival line-ups are typically decided with limited and unstructured feedback



Organizers lack a structured data model to capture preferences



Existing feedback channels (social media, comments) are noisy and non-quantifiable



No mechanism to translate user preferences into actionable insights

# Business Idea

A poll-based platform that structures user input into quantifiable data

Visitors influence the line-up through controlled voting mechanisms

Organizers obtain aggregated and comparable results to support booking decisions



# User journey- visitor



# User journey- organizer



# Core features



The platform allows users to suggest and vote for artists in a structured manner



Polls are generated based on predefined criteria and user input, ensuring relevance

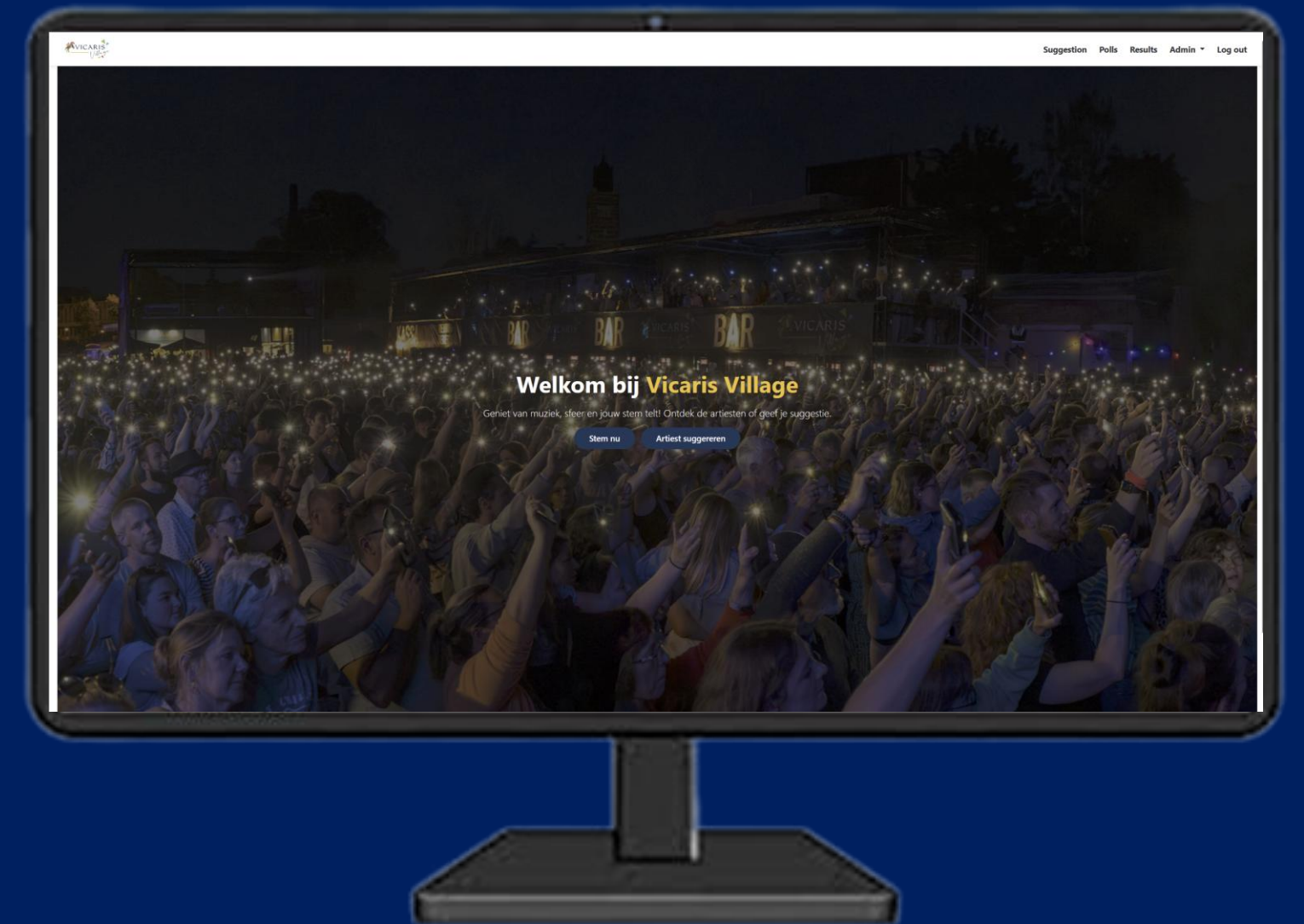
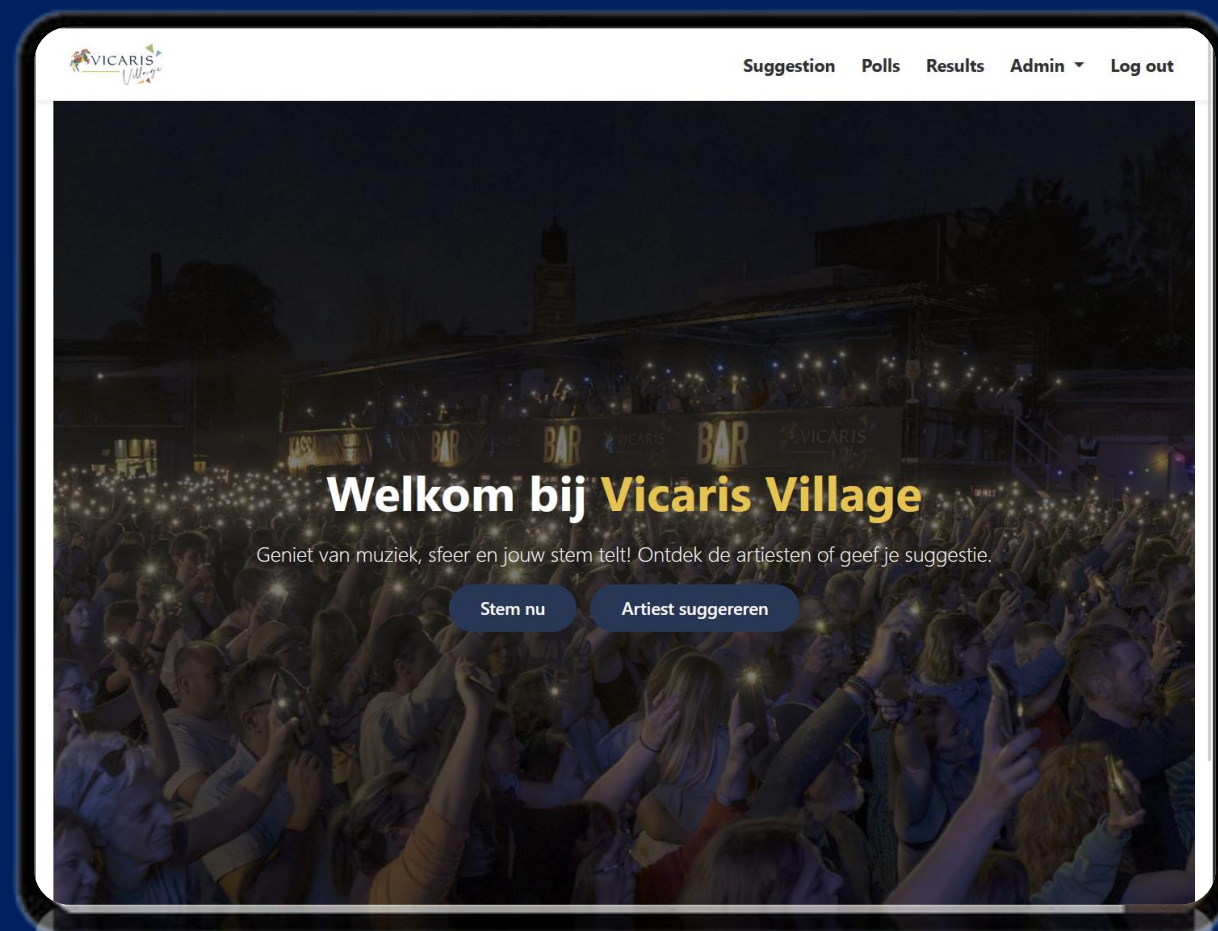


Voting results are automatically aggregated and stored consistently



Organizers can access an overview of poll outcomes through the application

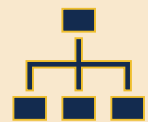
# Compatible on tablet and desktop



# Extra features



Administrators can manage artists and festival editions, including adding and removing entries



The application enforces role-based access, separating user and admin functionalities



Results are presented in a clear and interpretable format to support decision-making



The underlying data structure allows the platform to be reused for future festivals and editions

# Demo

<https://1drv.ms/v/c/2b685717d1176f31/IQBXTGSAPZziSo6jY0ZeQqT5ARf0T71pGdcN3ttObiWkPfU?e=5kPvPL>

# Thank you for listening

Group 38



*Herman Roelandt*  
PHOTOGRAPHY

# Appendix

- [Site Maspoe](#)
- [MVP render – polls](#)
- [Team 38 kanban - Miro](#) (not possible to edit)

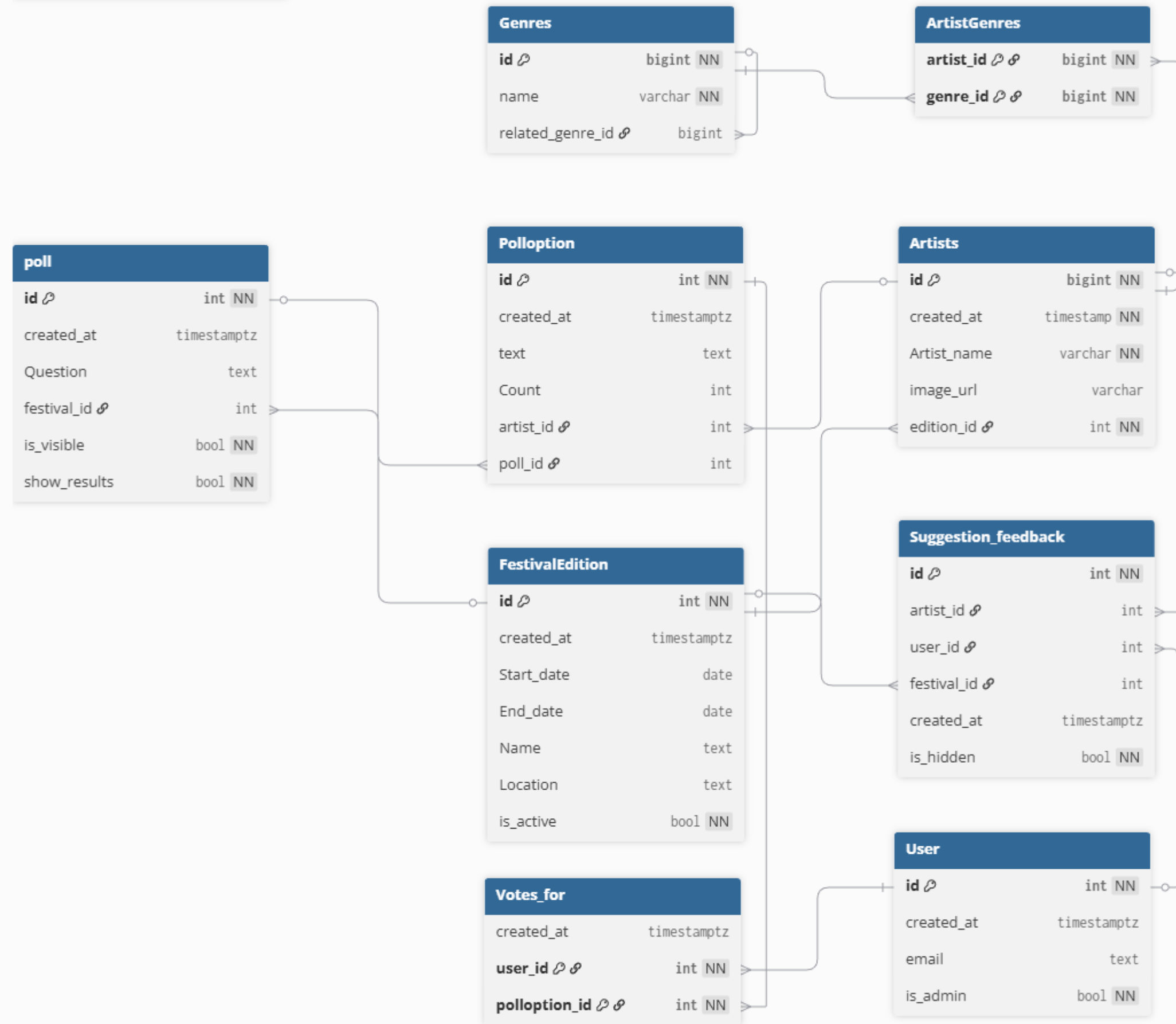
**Q&A**

# User stories

- As a user, I want to view available polls and submit a vote or suggestion, so that I can actively participate in the Vicaris Village selection process.
- Users can submit a vote via the polling interface
- Users can suggest an artist using a form
- As a user, I want to view clear and structured poll results and submitted suggestions, so that I can understand the collective preferences of the community.
- As an admin, I want to manage artists, editions and polls, so that the Vicaris Village application remains accurate, up to date and scalable for future events.

# ERD

- 9 entities
- 42 attributes
- Each entity has 1 or more primary keys
- Relationships implemented using foreign keys
- N:M relationships resolved using junction tables



# Overall architecture

- Flask *application factory* (`create_app()`)
- Central `models.py` using SQLAlchemy
- Blueprints in `app/routes/`
- HTML templates in `app/templates/`
- Static assets in `app/static/`
- Business logic in `app/services/`
- Helper utilities in `app/utils/`

# Flask app

- `create_app()` loads configuration (`Config`)
- Initializes database (`db.init_app`)
- Enables Flask-Migrate
- Registers all blueprints
- Context processors:
  - `current_user`
  - `poll_visibility`
  - `suggestions_visibility`
  - `results_visibility`
- `@app.before_request`-hook: always an active poll

# Routes

- `core.py` --> homepage
- `auth.py` --> register / login / logout
- `suggestions.py` --> artist suggestions (max. 5 per edition)
- `poll.py` --> poll loading, voting, and results
- `admin.py` --> admin dashboard
- `admin_editions.py` --> edition management

# Database models

- User --> authentication + is\_admin
- FestivalEdition --> edition details + active flag
- Artists — artists linked to an edition
- Genres + ArtistGenres --> many-to-many relationship
- SuggestionFeedback --> user suggestions per edition
- Poll, Polloption, VotesFor --> complete poll system

# Services and business logic

- `poll.py` service:
  - `get_active_festival()`
  - `get_or_create_poll_for_edition()`
  - `get_or_create_active_poll()`
- `genre_profile.py` service:
  - Builds user music profile from suggestions
  - Generates personalized poll options
- `session.py` utility:
  - `get_session_user()`

# Algorithmic support

- Builds a user genre profile based on suggestions
- Scores artists using genre similarity
- Selects top-ranked artists for personalised polls

# Strenghts

- Useful for festivals
- Easy to use for customers
- Easy to filter artists for admin
- Upscaleable
- Everyone can vote
- Sorted into genres

# Result

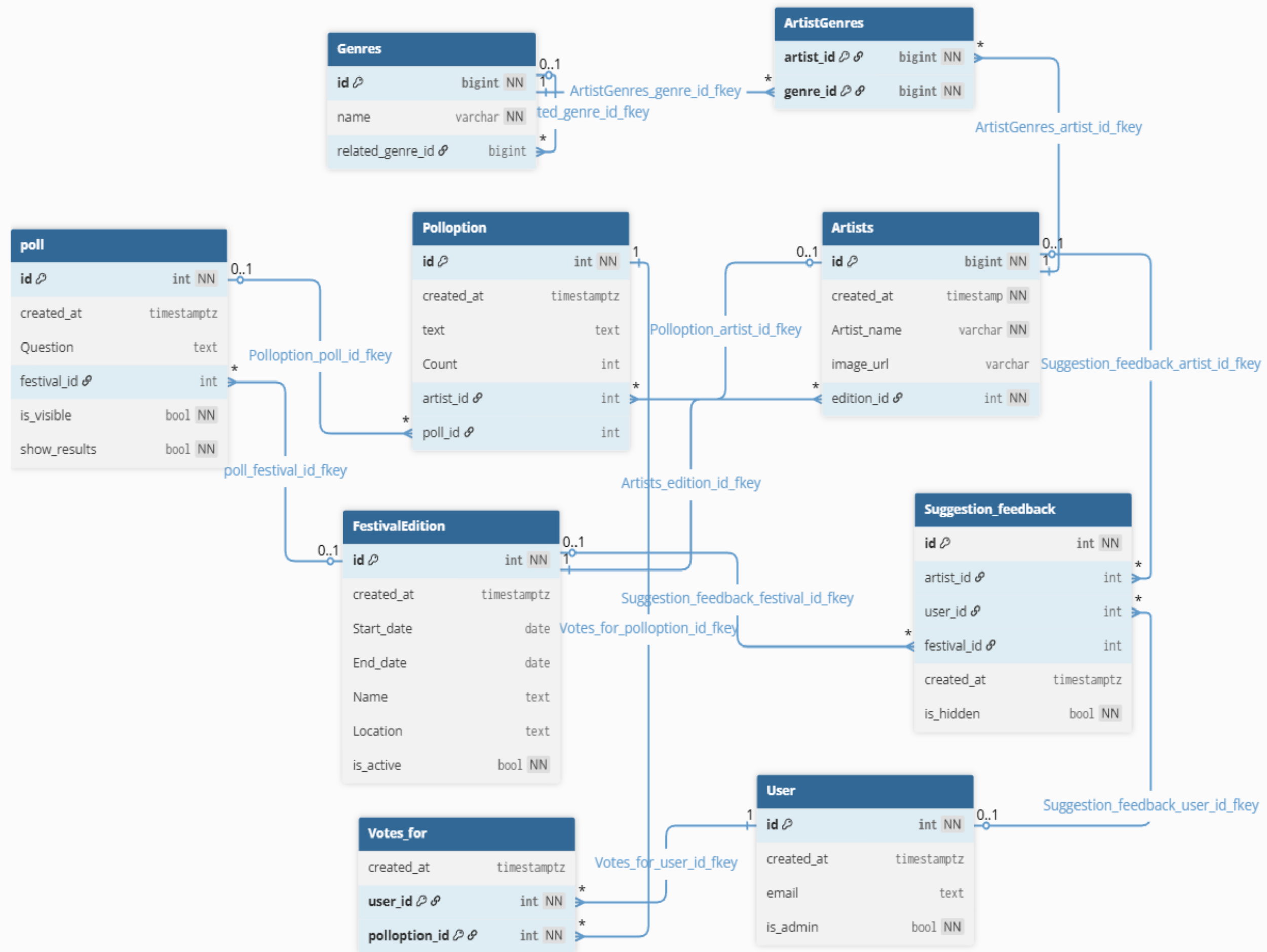


Tekstueel overzicht (toegankelijkheidsfallback)

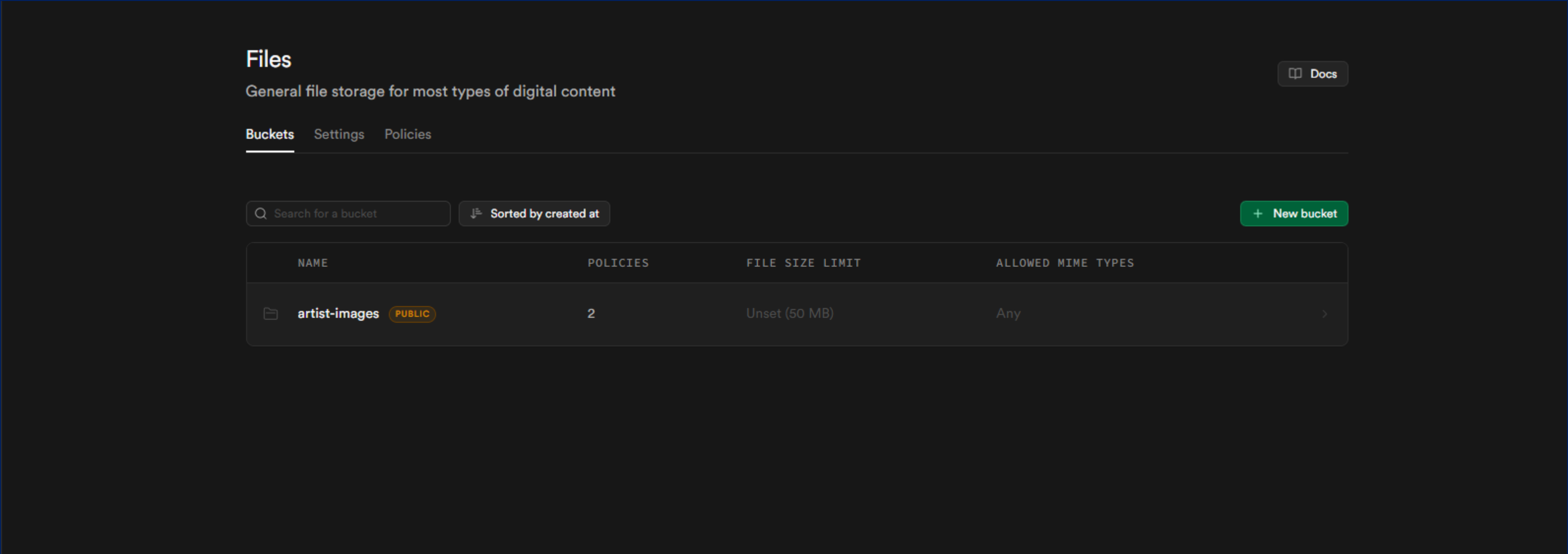
|                   |           |
|-------------------|-----------|
| Hip-Hop           | 14.3% (2) |
| Pop               | 14.3% (2) |
| Rock              | 14.3% (2) |
| Singer-Songwriter | 14.3% (2) |
| Vlaamse slagers   | 14.3% (2) |
| Alternative       | 7.1% (1)  |
| Electronic        | 7.1% (1)  |
| R&B               | 7.1% (1)  |
| Rap               | 7.1% (1)  |

# The future

- Making it more accessible for more festivals
- Making it easier for the organisers to edit
- Upscaling



# Bucket for images



# List of artist in supabase

## Stel een artiest voor

Kies jouw favoriete artiest uit de lijst hieronder en dien je suggestie in voor de huidige editie!

Kies artiest:

Angèle

Bazart

Black Box Revelation

David Guetta

Dua Lipa

Florence + The Machine

Greta Van Fleet

# Algorithmic Support

```
app > routes > poll.py > ...
137
138 @bp.get("/results")
139 def results():
140     user = get_session_user()
141     poll = get_or_create_active_poll()
142
143     # --- Muziekprofiel op basis van jouw suggesties (actieve editie) ---
144     genre_counts = {}
145     genre_percentages = {}
146
147     if user:
148         rows = (
149             db.session.query(Genres.name, func.count())
150             .join(ArtistGenres, ArtistGenres.genre_id == Genres.id)
151             .join(Artists, Artists.id == ArtistGenres.artist_id)
152             .join(SuggestionFeedback, SuggestionFeedback.artist_id == Artists.id)
153             .filter(SuggestionFeedback.user_id == user.id)
154             .filter(
155                 SuggestionFeedback.user_id == user.id,
156                 SuggestionFeedback.festival_id == poll.festival_id,
157             )
158             .group_by(Genres.name)
159             .all()
160         )
161         genre_counts = {g: c for g, c in rows}
162         total = sum(genre_counts.values()) or 1
163
164         genre_percentages = {
165             g: round(c * 100 / total, 1)
```

# Algorithmic Support

```
app > routes > poll.py > results
139 def results():
140     genre_percentages = {
165         g: round(c * 100 / total, 1)
166         for g, c in genre_counts.items()
167     }
168
169     # 🔥 Sorteer percentages aflopend
170     sorted_genres = sorted(
171         genre_percentages.items(),
172         key=lambda x: x[1],
173         reverse=True
174     )
175
176     # --- Jouw stemmen binnen de actieve editie ---
177     user_votes = []
178     if user and poll:
179         user_votes = (
180             db.session.query(VotesFor, Polloption, Poll, FestivalEdition, Artists)
181             .join(Polloption, VotesFor.polloption_id == Polloption.id)
182             .join(Poll, Polloption.poll_id == Poll.id)
183             .join(FestivalEdition, Poll.festival_id == FestivalEdition.id)
184             .join(Artists, Polloption.artist_id == Artists.id)
185             .filter(
186                 VotesFor.user_id == user.id,
187                 Poll.festival_id == poll.festival_id,
188             )
189             .order_by(FestivalEdition.Start_date.desc())
190             .all()
191         )
192
```

# Algorithmic Support

app > services > genre\_profile.py > generate\_poll\_for\_user

```
29 def generate_poll_for_user(user_id: int, num_options: int = 5):
30     """Generate a personalized set of poll options for a user."""
31     profile = get_user_genre_profile(user_id)
32     proximity_scores = genre_proximity_scores(profile.keys())
33     genre_id_lookup = {
34         name: gid for gid, name in Genres.query.with_entities(Genres.id, Genres.name).all()
35     }
36
37     rows = (
38         db.session.query(
39             Artists.id,
40             Artists.Artist_name,
41             Genres.name.label("genre"),
42         )
43         .join(ArtistGenres, ArtistGenres.artist_id == Artists.id)
44         .join(Genres, Genres.id == ArtistGenres.genre_id)
45         .all()
46     )
47
48     artist_data = {}
49     for artist_id, artist_name, genre in rows:
50         if artist_id not in artist_data:
51             artist_data[artist_id] = {
52                 "artist": artist_name,
53                 "genres": [],
54             }
55             artist_data[artist_id]["genres"].append(genre)
```

# Algorithmic Support

```
app > services > genre_profile.py > generate_poll_for_user
29 def generate_poll_for_user(user_id: int, num_options: int = 5):
56
57     scored = []
58     for artist_id, data in artist_data.items():
59         genre_score = sum(profile.get(g, 0) for g in data["genres"])
60
61         # Calculate how close this artist's genres are to the user's favourites.
62         proximity = max(
63             (
64                 proximity_scores.get(genre_id_lookup.get(genre_name, -1), 0.0)
65                 for genre_name in data["genres"]
66             ),
67             default=0.0,
68         )
69
70         # Did user suggest it?
71         self_suggested = (
72             SuggestionFeedback.query
73             .filter_by(user_id=user_id, artist_id=artist_id)
74             .first()
75             is not None
76         )
77
78         score = 3 * int(self_suggested) + 2 * genre_score + 5 * proximity
79
80         scored.append((score, artist_id, data["artist"]))
81
82     scored.sort(reverse=True)
```